Modern Tools in Machine Learning.

Samuel Schepp
Technische Hochschule Mittelhessen
Fachbereich MNI
Master-Seminar
samuel.schepp@mni.thm.de

Abstract

This paper introduces common machine learning tools like TensorFlow, PyCharm, and PyPlot, which are used to setup and run machine learning tasks with neural networks. A common "Hello World" script will be developed to show the concept and ideas of these tools. It will make use of the "Fashion MNIST dataset" by Zalando, which will be explored in detail.

In addition, there will be shown, how to run trainings on a graphics card and what TensorFlow is not doing.

1 MNIST Datasets

To get started with machine learning, there needs to be data that a model can read and produces a result of. In addition, a model needs much more data to train. In the "Hello World" example, which will be developed later, there will be used the "Fashion MNIST dataset", which is created by Zalando, an online fashion store. Zalando is developing solution, which can detect and classify pictures of cloth. Their current focus is machine learning.

A MNIST dataset contains four individual sets. The first set is a list of images, which will be used to train on. The second set is a list of class names, that refers to the actual thing, that is shown on the image at the same index. That means, that image 4 might show a jacket, the class name on set 2, index 4 says "jacket". These two datasets can now be used to train a model, which should recognize the image.

The remaining two datasets are again pictures and class names of cloth. These are used to test the model. The images are new and different ones, to make sure, the model is not over fitting. Over fitting means, that the model is only capable of recognizing the given dataset, but is likely to fail on unknown and new pictures.

1.1 Origin

The acronym "MNIST" database stands for "Modified National Institute of Standards and Technology database". It is a large database of handwritten digits, that is used to train neural networks and image processing systems. There are 60,000 training and 10,000 testing images, which are normalized, centered and 28 by 28 pixel in size. It is easy for convolutional neural networks, to classify these images at a failure rate of under 1 percent.

The images are encoded in a simple byte-array based format, so no conventional image application can open them. TensorFlow is able to decode the images and use them as input, since it became a common format.

Figure 1: The original MNIST database with handwritten digits

1.2 Examples

Since the MNIST database is popular amongst machine learning scientists, it became a stan-

dard format for other content. Zalando uses it, to provide example data to the public and try out different learning systems.

They created a database which contains clothing and its names (class). Figure 1.2 shows an extract of the dataset. It has a size of 26 MB for training and 4.3 MB for test samples. There are 10 classes of clothing: T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag and ankle boot. It is not a coincidence, that the count of classes here is the same as the original set, which contains handwritten digits. A system, that can train the handwriting, can also train and recognize the clothing, without any reimplementation.

Figure 2: Snippet of the Zalando fashion MNIST.

TensorFlow is able to directly load the fashion MNIST, since it is part of the python package.

```
from tensorflow import keras

fashion_mnist = keras.datasets.
    fashion_mnist
(train_images, train_labels), (test_images
    , test_labels) = fashion_mnist.
    load_data()
```

Listing 1: Loading the fashion MNIST in TensorFlow (Python)

The tuples contain TensorFlow-friendly data, that can directly be used in Keras, which is TensorFlow‘s high level API to configure and run neural networks.

2 TensorFlow

TensorFlow is a software platform for numerical computation, with focus on neural networks and deep learning. It is developed by Google’s AI organization and provides frameworks for many scientific domains.

It provides hardware acceleration for graphics cards and it own chips called ”TensorFlow Processing Unit” (TPU).

Most of the time, the Python package is adequate to perform many basic and advanced machine learning tasks.

2.1 Keras

The Phyton package of TensorFlow contains Keras, which is a high level interface to several deep learning back ends, like Theano, Microsoft Cognitive Toolkit and TensorFlow.

It is the default API for TensorFlow, although it is a independent open source project. That means you will be able to run deep learning algorithms, that are developed for TensorFlow, on different frameworks.

2.2 TensorFlow.js

To train and deploy models for the browser or Node.js, there is the TensorFlow.js JavaScript package. It provides the common low-level algebra library and a high level, Keras-like API for building and running neural networks. In addition, it allows users to convert pre-existing TensorFlow models, to run them in the browser or in a Node.js environment.

This allows JavaScript applications to directly integrate machine learning capabilities into their stack and out source computing power to the client devices, instead of the server.

2.2.1 Browser

The framework can be used directly as a script import from the CDN repository ”https://www.jsdelivr.com”,

```
<script src="https://cdn.jsdelivr.net/npm/
    @tensorflow/tfjs@0.14.1/dist/tf.min.js"
    ></script>
```

Listing 2: Loading TensorFlow.js in the browser via CDN

Now, the Keras API is available for usage and a model can be trained or ran on the

browser's JavaScript engine. Hardware acceleration is available via WebGL, which provides a run performance, that is only two times slower then the native Python implementation. For large models, training is up to 15 times slower.

2.2.2 Node.js

For Node.js, there is another package available, which is called "tfjs-node". It uses the native C++ bindings of TensorFlow, but provides the same interface as the browser version. This enables developers to interchange TensorFlow code between browser and server, without any rewrites.

To use the package, install it via NPM.

```
npm install @tensorflow/tfjs-node
```

Listing 3: Installing TensorFlow.js for Node.js

The package has access to all of the common hardware acceleration interfaces of the operating system, like NVIDIA CUDA, which allows models to train and run on a graphics card. That means, that is is possible to write JavaScript neural network code, which runs with maximum performance.

2.3 TensorFlow lite

To use models in mobile applications, there is the "TensorFlow Lite" library, which can be integrated in to existing native iOS and Android mobile applications. It is a C++ library, which comes with native API bindings for the respecting development environments.

Models, that are created in the classic TensorFlow python environment have to be converted to a lightweight and flattened model, that can be shipped with the mobile application.

Even complex models, that are used to classify images in 1000 categories are only 17 MB in size.

The key requirement of a mobile application that uses machine learning is speed and battery performance. TensorFlow Lite is optimizing models by making a trade off between model complexity and size. The tools are helping developers to fine tune the model size, by allowing adjustments even while converting the model.

A common image classification model's accuracy A is growing with file size D by a factor of $H = \sqrt{D}$. As a consequence, the models file size can be optimized at first, without compromising much performance. A model can also be optimized by avoiding over fitting and training with distinguished and useful data. However, these steps require adequate scientific approaches.

2.3.1 Usage example iOS

To use TensorFlow Lite models in an iOS applications, the C++ library has to be built from source and linked against. This is achieved by cloning the repository and running some shell scripts in the directory. A library is built, which is called "libtensorflow-lite.a". The iOS project now has to be configured with the correct library and header search paths.

The API is a low level interface and some boilerplate code is needed to make a simple image recognition demonstration working. To classify an image, the model has to run against a pixel buffer `CVPixelBufferRef` and the result will be written into a vector of class labels and their probability `std::vector<std::pair<float, int>>`. This requires about 100 lines of objective C and C++ code.

For a more simple API, Apple created CoreML, which is yet restricted to common use cases like image and text classification. It supports TensorFlow lite models as base format.

2.4 Limits

Originally, TensorFlow was developed to create an open-source data flow oriented programming library. The goal was to enable symbolic math operations, lazy evaluation and hardware acceleration, which runs platform independent.

A simple multiplication of two constants can be run with the following Python code.

```
sess = tf.Session()
x = tf.constant(3, dtype=tf.int8)
y = tf.constant(2, dtype=tf.int8)
z = tf.multiply(x, y)
sess.run(z)
# 6
```

Listing 4: Multiplying two constants in Tensorflow

The operation are represented in a graph, which will be evaluated, when the session in ran.

Neural networks are only a part of TensorFlow, which evolved to the most important aspect, since the maintainers are promoting almost exclusively the machine learning facets of TensorFlow.

With this idea behind TensorFlow, it is important to known, what it is *not* doing. It is not designed to support data scientist in collecting, managing, preprocessing and analyzing data and model results. It is not possible, to let TensorFlow make semantic inferences on the inputs or outputs. It only runs operations of maths and information science, which makes it low level on this point of view.

3 PyPlot

To be able to work with data and analyze results, PyPlot offers a Python framework to visualize data directly in the IDE. It can be used in the TensorFlow-scripts and output images, graphs, tables and diagrams. The API is designed to use a MATLAB-like interface and enables a wide variety of customization options.

Listing 5 generates a plot of two arrays, which represent the sinus function, by plotting two arrays, that contain discrete example values. Variable x is an array that is generated by creating values from 0 to 2π with an interval of 0.5, which results in 13 values. y is the result of mapping y using the sinus function. Running the script in an supported IDE like Jetbrains PyCharm, enables interactive display of the generated plots by calling plt.show().

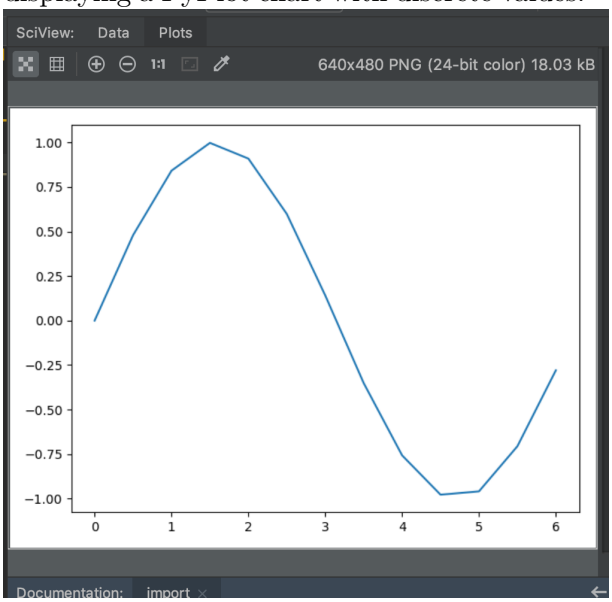
```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 2*np.pi, 0.5);
y = np.sin(x)
plt.plot(x, y)
plt.show()
```

Listing 5: Generating a plot of sin(x) with some discrete values.)

Figure 3 shows a screenshot of the PyCharm IDE. The window SciView displays all the plots, that are generated while the code is ran.

This helps to understand machine learning algorithms, since it can show input and output data. Image recognition algorithms can output the image first, then the processed one and then the result after the session finished. An examples can be found in section 5, where there will the results be analyzed using PyPlot. Figure 1.2 is generated using PyPlot.

Figure 3: Screenshot of Jetbrains PyCharm displaying a PyPlot chart with discrete values.



4 Hardware Acceleration

When using Tensorflow's high-level APIs like Keras, most of the optimizations are already included. There are some ways, a developer can still improve performance.

4.1 CPU

When using CPU specific improvements like Intel MKL-DNN[1] or targeting special architectures like Xeon Phi, TensorFlow should be built from source, so that all of the supported instructions are available.

Since different kind of neural networks require other optimizations, a general performance guide is not possible. For Example: When training CNN[2] based models, TensorFlow should be built with MKL, so that AVX[3] is available. That way, modern consumer processors from Intel are high-performing and can train CNN models up to 9 times faster[4]. There are many configuration options available, which can improve performance for special setups even further.

[1] Intel Math Kernel Library for Deep Neural Networks (Intel MKL-DNN)

[2] Convolutional Neural Networks

[3] Advanced Vector Extensions

[4] https://towardsdatascience.com/stop-installing-tensorflow-using-pip-for-performance-sake-5854f9d9eb0c

4.2 GPU

Using a GPU can also increase performance drastically. NVIDIA CUDA[5] is a parallel computing platform and programming interface to abstract parallelism, which lets users run single threaded tasks on the CPU and multi-threaded task on the GPU. TensorFlow takes advantage of CUDA and optimizes training sessions to run multi-threaded. Other libraries like NumPy are also optimized , so that pre- or post-processing can be sped up too. In some situations, a consumer grade graphics card can be up to 15 times faster[6] than a consumer grade desktop processor.

At the moment, CUDA is the only hardware platform, that is support officially in TensorFlow's GPU package. It is possible to use external GPUs via macOS's eGPU interface, which requires to disable macOS's System Integrity Protection.

4.3 Device selection

If enabled, TensorFlow will run all of it's computations on the GPU, otherwise it will fallback to the CPU. Running something on selected devices can be accomplished by using a 'with' block.

```
with tf.device('/cpu:0'):
  a = tf.constant([1.0, 2.0, 3.0, 4.0,
    5.0, 6.0], shape=[2, 3], name='a')
  b = tf.constant([1.0, 2.0, 3.0, 4.0,
    5.0, 6.0], shape=[3, 2], name='b')
c = tf.matmul(a, b)
```

Listing 6: Running the instantiation on the CPU, the multiplication on the GPU.

In listing 6, the allocation and configuration of the matrix and assignment to a and b are done on the CPU. In Python, the 'with' keyword provides an object to a block. it is used here to define the device for the statements.

The actual matrix multiplication is done on the GPU, because it is the default device.

5 Hello World

References

Fashion-MNIST

https://github.com/zalandoresearch/fashion-mnist

Accessed: 2019-01-12.

Zalando Research

https://research.zalando.com

Accessed: 2019-01-12.

MNIST Database

http://yann.lecun.com/exdb/mnist/

Accessed: 2019-01-12.

TensorFlow Processing Unit

https://cloud.google.com/tpu/?hl=de

Accessed: 2019-01-12.

TensorFlow Lite

https://www.tensorflow.org/lite/performance/best_practices

Accessed: 2019-01-15.

Mobilenet_v2 Model

https://github.com/tensorflow/models/blob/master/research/slim/nets/mobilenet_v1.md

Accessed: 2019-01-15.

TensorFlow performance test: CPU VS GPU

https://medium.com/@andriylazorenko/tensorflow-performance-test-cpu-vs-gpu-79fcd39170c

Accessed: 2019-03-06.

Stop Installing Tensorflow using pip for performance sake!

https://towardsdatascience.com/stop-installing-tensorflow-using-pip-for-performance-

Accessed: 2019-03-06.

Training Your Neural Net with eGPU Acceleration on Mac with Tensorflow 1.5

https://medium.com/@jianshi_94445/training-your-neural-net-with-egpu-acceleration-on-ma

Accessed: 2019-03-06.

[5] Compute Unified Device Architecture

[6] https://medium.com/@andriylazorenko/tensorflow-performance-test-cpu-vs-gpu-79fcd39170c